

08. gyakorlat - For ciklusok, keresés és függvények

Ebben a gyakorlatban az elágazások sorrendje, a for ciklus kétféle használata, számlálás, korai return(), karakterek feldolgozása és egyszerű prímtesztelés kerül elő.

A tanári változat nemcsak a várható eredményt írja le, hanem azt is, milyen R-szabályt érdemes a kódból felismerni. A véletlen mintáknál konkrét szám helyett a működést és a várható mintázatot emeli ki.

Az elágazások sorrendje: BIMM-BAMM

Ha több feltétel egyszerre igaz lehet, az if - else if láncban az első igaz ág fut le, a későbbiek már nem vizsgálja.

```
jatekl=function(n){  
  if(n%%3==0){  
    print("BIMM")  
  }  
  else if(n%%5==0){  
    print("BAMM")  
  }  
  else if(n%%15==0){  
    print("BIMMBAMM")  
  }  
}  
jatekl(4)  
jatekl(12)  
jatekl(25)  
jatekl(45)
```

Magyarázat / tanulság

4-re nincs kiírás. 12-re BIMM, 25-re BAMM. 45 osztható 15-tel is, de már az első, 3-mal való oszthatósági feltétel igaz, ezért BIMM lesz. A 15-ös ág ebben a sorrendben soha nem érhető el 15 többszöröseinél.

```
jatek2=function(n){  
  if(n%15==0){  
    print("BIMMBAMM")  
  }  
  else if(n%5==0){  
    print("BAMM")  
  }  
  else if(n%3==0){  
    print("BIMM")  
  }  
}  
jatek2(4)  
jatek2(12)  
jatek2(25)  
jatek2(45)
```

Magyarázat / tanulság

A speciálisabb, 15-tel osztható eset kerül előre, ezért 45-re már BIMMBAMM jelenik meg. 4 továbbra sem ír ki semmit; 12 BIMM, 25 BAMM.

For ciklus

A for(valtozonev in vektor) sorra végigveszi a vektor elemeit, és minden elemre lefuttatja a blokkot.

```
for(valtozonev in vektor){  
  SZÁMOLÁS  
}
```

```
for(i in 1:10){  
  print("alma")  
}
```

Magyarázat / tanulság

Az i sorra felveszi az 1,2,...,10 értékeket, de a blokk nem használja i-t, ezért az "alma" szó tízszer íródik ki.

```
for(i in 34:54){  
  print(i)  
}
```

Magyarázat / tanulság

Az i 34-től 54-ig halad, és minden körben az aktuális értéket írja ki. Összesen 21 szám jelenik meg.

Páros elemek megszámlálása

Ugyanazt a feladatot először vektoros indexeléssel, majd kétféle for ciklussal oldjuk meg.

```
x=sample(1:1000,80,replace=F)
length(x[x%%2==0])
```

Magyarázat / tanulság

80 különböző véletlen számot választ 1 és 1000 között. Az `x%%2==0` logikai maszk kiválasztja a párosakat, a `length()` pedig megszámolja őket. A véletlen minta miatt az eredmény futtatásonként változhat.

```
paros1=function(vektor){
  count=0
  for(k in vektor){
    if(k%%2==0){
      count=count+1
    }
  }
  return(count)
}
paros1(x)
```

Magyarázat / tanulság

A ciklus közvetlenül a vektor értékein halad végig. Minden páros elemnél eggyel növeli a `count` számlálót, majd visszaadja a darabszámot.

```
paros2=function(vektor){
  count=0
  n=length(vektor)
  for(i in 1:n){
    if(vektor[i]%%2==0){
      count=count+1
    }
  }
  return(count)
}
paros2(x)
```

Magyarázat / tanulság

Itt a ciklus nem az értékeken, hanem az `1:n` indexeken halad. Az aktuális elem a `vektor[i]`. Ugyanarra az `x`-re a `paros1`, `paros2` és a vektoros `length()` azonos eredményt kell adjon.

Kereskedő bevétele

A példában a vektor kumulált bevételt tartalmaz. Egy nap akkor számít új bevételt termelő napnak, ha az érték nagyobb az előző napi értékénél; az első nap külön eset.

```
y=c(1000,1500,1500,2000,4000,5100,5100,5100,5400,6800,7000,7000,8700)
```

Magyarázat / tanulság

Létrehozza a példában használt, nem csökkenő kumulált bevétel-vektort.

```
bevetel=function(vektor){  
  if(vektor[1]>0){  
    count=1  
  }  
  else if(vektor[1]==0){  
    count=0  
  }  
  n=length(vektor)  
  for(i in 2:n){  
    if(vektor[i]-vektor[i-1]>0){  
      count=count+1  
    }  
  }  
  return(count)  
}  
bevetel(y)
```

Magyarázat / tanulság

Az első napot 1-nek számolja, ha már pozitív a bevétel. Ezután minden pozitív napi növekménynél emeli a számlálót. A megadott y-ra az eredmény 9. A kód a feladat szerinti nemnegatív bemenetre készült; negatív első elemnél a count nem kapna értéket.

print() és return() ciklusban

A print() csak kiír, a return() viszont az egész függvény futását azonnal befejezi.

```
proba=function(){  
  for(i in 1:10){  
    if(i>4){  
      print(i)  
    }  
  }  
}  
proba()
```

Magyarázat / tanulság

A ciklus végigfut 1-től 10-ig, és 5,6,7,8,9,10 kerül kiírásra. A print() után a függvény folytatódik.

```
proba=function(){  
  for(i in 1:10){  
    if(i>4){  
      return(i)  
    }  
  }  
}  
proba()
```

Magyarázat / tanulság

A függvény az első $i > 4$ esetnél, vagyis $i = 5$ -nél azonnal kilép. Eredménye 5; 6-tól 10-ig már nem jut el.

Palindromvizsgálat

Palindrom az a szó, amely előlről és hátulról ugyanaz. A kód a karaktereket vektorrá bontja, majd tükörpozíciókat hasonlít össze.

```
palindrom=function(szó){  
  betuk=unlist(strsplit(szó,""),use.names=F)  
  n=length(betuk) #nchar(szó)  
  for(i in 1:floor(n)){  
    if(betuk[i]!=betuk[n+1-i]){  
      return("Nem palindrom")  
    }  
  }  
  return("Ez a szó palindrom")  
}
```

Magyarázat / tanulság

A `strsplit(...,"")` karakterekre bontja a szót, az `unlist` vektorrá alakítja az eredményt. Ha bármelyik tükörpár eltér, azonnal "Nem palindrom" tér vissza. A ciklus az egész szó hosszáig fut, ezért a párokat részben kétszer ellenőrzi; elég lenne a szó feléig is, de a forráskód így is működik a tipikus bemeneteken.

```
palindrom("létkoktél")  
palindrom("alma")  
palindrom("abczuorcba")
```

Magyarázat / tanulság

A "létkoktél" palindrom, az "alma" nem. Az "abczuorcba" sem palindrom, mert valamelyik tükörhelyen eltérő betűk találhatók.

Prímtesztelés

Egy összetett számnak van olyan osztója, amely legfeljebb a négyzetgyöke. Ezért nem kell $n-1$ -ig próbálkozni.

```
prime=function(n){  
  ind=T  
  m=floor(sqrt(n))  
  for(i in 2:m){  
    if(n%%i==0){  
      ind=F  
    }  
  }  
  if(ind==T){  
    print("Ez a szám prímszám")  
  }  
  else if(ind==F){  
    print("Ez a szám nem prímszám")  
  }  
}
```

Magyarázat / tanulság

Az ind kezdetben TRUE. 2-től floor(sqrt(n))-ig keres osztót; ha talál, FALSE lesz. A kód 4-nél nagyobb pozitív egészekre jól szemlélteti az elvet, de n=2 vagy n=3 esetén a 2:1 sorozat miatt hibás eredményt adna; ez egy fontos szélső eset.

```
prime(42)  
prime(151)  
prime(21353243)  
prime(213532432)  
sqrt(213532432)
```

Magyarázat / tanulság

42 nem prím; 151 prím; 21353243 prím; 213532432 páros, tehát nem prím. A sqrt() megmutatja, meddig kell legfeljebb osztókat keresni.